Pretende-se desenvolver, recorrendo à biblioteca Winsock, uma aplicação Win32, em modo consola, que execute a seguinte sequência de ações:

- Cria um *socket* TCP designado s;
- Associa s1 ao porto de escuta “6060”;
- Entra num ciclo em que vai aceitando pedidos de ligação TCP em s;
- Durante esse ciclo, sempre que uma ligação TCP é aceite, acrescenta o *socket* correspondente à tabela SOCKET partners[10];
- Termina o ciclo quando ocorre um *timeout* de dez segundos ou a tabela partners está totalmente preenchida. O *timeout* deve ser implementado recorrendo à função select;
- Envia a, a cada um dos destinos registados na tabela partners, a *string* “EXEC x\r\n”, sendo *x* o número de endereços registados na tabela;
- Entra num ciclo infinito, em que recorre a uma variável do tipo fd_set, devidamente, configurada e à função select para aguardar pela receção de caracteres em qualquer um dos *sockets* existente na tabela partners;
- Sempre que recebe um carácter, imprime-o na saída standard;
- A aplicação termina se ocorrer um *timeout* de receção de dez segundos na função select;
- Durante a execução, qualquer erro faz com que a aplicação termine imediatamente.

Complete as partes em falta no código fonte. Deve identificar as alíneas ([a], [b], …, [i]) correspondentes às respostas que apresentar.

```c
#pragma comment(lib, "Ws2_32.lib")

#include <stdio.h>
#include <winsock.h>

#define LISTENING_PORT 6060
#define TIMEOUT       10 //sec
#define BUFFERSIZE 1000
#define EXEC "EXEC"

void Abort(char* msg, SOCKET s);

//****************************************

int main(int argc, char* argv[]) {

        SOCKET s = INVALID_SOCKET, cli;
        int i, continua;
        int iResult, nbytes, count;
        struct sockaddr_in addr;
        char buffer[BUFFERSIZE];
        WSADATA wsaData;
        SOCKET partners[10];
        struct timeval timeout = {TIMEOUT,0}; //10 segundos
        fd_set conjunto;

        iResult = WSAStartup(MAKEWORD(2, 2), &wsaData);
        if (iResult != 0) {
                printf("WSAStartup failed: %d\n", iResult);
                exit(EXIT_FAILURE);
        }

/* a) Cria o socket TCP s */

        if (/* ... */) Abort("Impossibilidade de abrir o socket", s);


/* b) Associa s ao porto LISTENING_PORT */

        memset((char*)&addr, 0, sizeof(addr));
        addr.sin_family = AF_INET;
        /* ... */

        /* ... */
                Abort("Impossibilidade de associar-se ao porto de escuta", s);

        if(listen(s, 5) == SOCKET_ERROR)
                Abort("Impossibilidade de registar-se para escuta", s);

        FD_ZERO(&conjunto);
        continua = 1;

        for (count = 0; count < 10; count++) {

/* c) Recorre à função select para aguardar que haja um pedido de ligação em s até um limite
definido pela variável timeout (já iniciada na declaração)*/

                FD_SET(/* ... */, &conjunto);

                switch (select(/* ... */, &timeout)) {

                /* ... */:
                        Abort("Erro no select", s);
                        break;
                /* ... */:
                        continua = 0; //timeout
                        break;

                /* ... */:
                        break;
                }

                if (!continua) //timeout -> sai do ciclo
                        break;

/* d) Aceita uma ligação TCP em s. Não é necessário obter o endereço do socket remoto. */

                cli = /* ... */
```

```
                if (cli == SOCKET_ERROR)
                        Abort("Erro na recepcao de datagramas", s);

                partners[count] = cli;
        }

        if (count == 0) {
                printf("Sem qualquer ligacao estabelecida!");
                return EXIT_SUCCESS;
        }

        sprintf_s(buffer, sizeof(buffer), "%s %d", EXEC, count);

/* e) Envia, através de cada um dos sockets TCP existentes na tabela partners, a string colocada
em buffer. Termina a aplicação invocando a função Abort, se ocorrer qualquer tipo de erro*/

/* ... */

        FD_ZERO(&conjunto);

/* f) Coloca, na variável conjunto (do tipo fd_set), os sockets TCP existentes na tabela
partners*/

        /* ... */

        FD_SET aux;
        char c;
        continua = 1;
        while(continua) {

                aux = conjunto;

/* g) Recorre à função select para aguardar que haja dados prontos a serem recebidos, em qualquer
um dos sockets da tabela partners, até um limite definido pela variável timeout (já iniciada na
declaração). Recorre, para o efeito, à variável aux, já iniciada. */

                switch (select(/* ... */)) {

                /* ... */:
                        Abort("Erro no select", s);
                        break;

                /* ... */:
                        continua = 0; //timeout
                        break;

                /* ... */:
                        for (i = 0; i < count; i++) {

/* [h] Verifica se o socket, no índice i da tabela partners, está pronto para leitura (ou seja, se
uma operação de leitura não irá bloquear). */

                                if (/* ... */) {

/* [i] Recebe um único caractere no socket guardado no índice i da tabela partners, colocando-o
diretamente na variável c. */
                                        if (/* ... */)
                                                Abort("Erro de rececao", s);
                                        putchar(c);
                                } //if
                        } //for
                } //switch
        }

        closesocket(s);
        WSACleanup();

        return EXIT_SUCCESS;
}

// ************************************************************************

void Abort(char* msg, SOCKET s) {
        fprintf(stderr, "Erro fatal: <%s> (%d)\n", msg, WSAGetLastError());
        if (s != INVALID_SOCKET)
                closesocket(s);
        WSACleanup();
        exit(EXIT_FAILURE);
}
```